COMANDOS BÁSICOS, COMODINES Y AGRUPAMIENTO DE COMANDOS EN LINUX

SALIDA DEL SISTEMA

El apagado de LINUX es un proceso que normalmente lo hace el **root**.

Como LINUX es un s.o. multitarea, las lecturas y escrituras de disco se realizan en una memoria intermedia (memoria principal o RAM). Cuando escribimos en el disco, realmente lo hacemos en la memoria, y hasta que no sea estrictamente necesario no se volcarán los datos de la memoria al disco. Esto mejora el rendimiento del sistema, ya que los accesos a disco son muy lentos. Si se va la luz repentinamente o si apagamos el ordenador pulsando el interruptor de Power, los datos de la memoria no se llegan a volcar a disco, por lo que se pueden perder o "corromper".

Para garantizar la integridad del sistema, y que los discos se sincronicen, hay que hacer lo que se llama un **apagado seguro**. El comando para hacerlo es:

```
# shutdown –h now
```

El argumento now, especifica que se cierre el sistema inmediatamente (esto significa que podríamos avisar a otros usuarios de que dentro de 5 minutos, por ejemplo, se apagará el sistema). El modificador –h indica que se pare el sistema después del cierre. Cuando veamos el mensaje "System halted", podemos apagar el ordenador.

```
# shutdown –r now
```

El modificador –r indica un RESET seguro (la forma elegante del # reboot)
Existen algunos otros comandos relacionados con el apagado del sistema, que en realidad son variantes del comando shutdown:

- # **halt**: Detiene también el sistema. Es parecido a shutdown.
- # **reboot**: Es el comando para hacer un CTRL-ALT-SUPR. Cierra el sistema y lo vuelve a iniciar.

Además de estos comandos hay otros que tienen que ver con la finalización de una sesión de usuario. Estos son:

- # **logout**: Finalizar la sesión actual.
- # **exit**: Finaliza la ejecución del shell actual, que muchas veces suele corresponder con la sesión de un usuario
- # **CTRL-d**: Igual que exit

SINTAXIS GENERAL DE UN COMANDO EN LINUX

La sintaxis general de un comando en LINUX es:

```
$ comando [-modificador/es] [argumento/s]
```

Los comandos se escriben siempre en minúsculas.

Los modificadores son opcionales. Suelen ser letras mayúsculas o minúsculas precedidos por un guión. Se puede utilizar más de un modificador con un mismo comando.

Los argumentos son opcionales. Suelen ser nombres de archivos o directorios generalmente. También cadenas de caracteres. Puede haber varios argumentos separados por espacios en blanco.

Los corchetes de la sintaxis general significan opcionalidad y NO SE DEBERÁN ESCRIBIR.

Ejemplos:

```
$ date
$ ls –l
$ ls –lar
$ cp alfa beta
$ echo "hola que tal"
$ rm –r /uno /dos /tres
```

METACARACTERES O COMODINES

En LINUX existen ciertos caracteres especiales que sirven para aplicar determinados comandos a archivos que cumplan los criterios especificados por esos caracteres.

En MS-DOS estos caracteres especiales se llamaban comodines y eran el asterisco * y la interrogación ?.

Ejemplos:

`dir *.txt` → muestra todos los archivos con extensión TXT

`dir ???.doc` →muestra aquellos archivos cuyo nombre tuviera 3 caracteres como máximo con extensión DOC

En LINUX los caracteres comodín o metacaracteres son:

INTERROGACION (?)

Sustituye a un carácter cualquiera en el nombre de un archivo o directorio.

Ejemplos:

`$ ls ???` si suponemos que estamos en el directorio /bin, mostrará los archivos/directorios que tengan 3 caracteres cualesquiera. Por ejemplo awk, bsh, cat, tar, vim.

ASTERISCO (*)

Sustituye a un número indeterminado de caracteres en el nombre de un archivo o directorio.

Ejemplos:

`$ ls a*` muestra los archivos/directorios que empiecen por a. Por ejemplo arch, awk, ash.static

`$ ls ?a*` muestra los archivos/directorios cuyo 2º carácter sea una a. Por ejemplo false, cat, date

`$ ls ??a*` muestra los archivos/directorios cuyo 3er carácter sea una a. Por ejemplo uname

`$ ls *` .. si suponemos que estamos en el directorio raíz se visualizarían en pantalla todos los nombres de archivos, directorios y subdirectorios.

`$ ls *a*` muestra archivos/directorios que contengan el carácter a. (Nota: Esto no se podía hacer en MS-DOS)

Caracteres entre corchetes [...]

Indica que cualquier archivo o directorio que contenga uno de los caracteres del corchete será tomado por válido.

Ejemplos:

`$ ls *[stv]` muestra archivos/directorios que terminan por s, t ó v. Por ejemplo, cat, mount, ls, sort

$ ls [afr]??[xy] muestra archivos/directorios que empiecen por a, f ó r, a continuación vengan dos caracteres cualesquiera y terminen por x ó por y.

Rango o intervalo de caracteres [car – car]

Indica un rango entre caracteres ASCII (la tabla de caracteres ASCII está en el Anexo 3). Cualquier archivo o directorio que contenga uno de los caracteres del rango será válido.

El carácter: **!** Indica invertir el rango que venga detrás.

Ejemplos:

$ ls *[a-j] muestra archivos/directorios que terminen por cualquier carácter comprendido entre la a y la j (es decir, a, b, c, d, e, f, g, h, i, j).

$ ls a[!1-5]* muestra archivos/directorios que empiecen por a, el segundo carácter sea uno cualquiera **que no esté** comprendido entre 1 y 5, y tenga un número indeterminado de caracteres a continuación.

$ ls p*[A-z] muestra archivos/directorios que empiecen por p y terminen por cualquier carácter comprendido entre la A y la z (es decir, todas las letras mayúsculas y minúsculas, mas los símbolos especiales que hay en medio como [,], \)

$ ls *p[aj-z] muestra archivos/directorios que terminen por p y a continuación venga uno de los caracteres del rango, que son a,j,k,l,m,n,....,z.

$ ls a[15-90]* muestra archivos/directorios que empiecen por a, el segundo carácter sea uno de los del rango, que son 1,5,6,7,8,9,0, y tengan un número indeterminado de caracteres a continuación. (NOTA: ¡Ojo que el segundo carácter NO ES cualquiera comprendido entre 15 y 90!)

$ ls a[1-3][0-9]* muestra archivos/directorios que empiecen por a, el segundo carácter sea un número entre 1 y 3, y el tercero , otro número entre 0 y 9, es decir, que empiecen por a y después tengan un número entre el 10 y el 39.

Secuencia de cadenas { cadena1, cadena2, …., cadena3 } o rango de valores {inicial..final}

- Permite indicar una serie de cadenas (separadas por comas) que pueden aparecer en el nombre del archivo:
  {Fichero, fichero, Carta, carta}[0-9]* → Archivos cuyo nombre empiece por Fichero o carta y luego tengan un dígito.
- También permite indicar un rango numérico o alfabético dentro de un nombre de archivo para referirse a muchos:
  touch Arch{A..H}.txt → Crea los archivos desde ArchA.txt hasta ArchH.txt
  rm fich{14..16}.txt → Borra los ficheros desde fich14.txt hasta fich16.txt

AGRUPAMIENTO DE COMANDOS

En LINUX se pueden introducir varios comandos en una misma línea de las siguientes formas:

$ comando1 ; comando2 ; comando3; ...

Se trata de una ejecución sucesiva e independiente de los comandos especificados. Primero se ejecutará comando1; cuando termine, comando2; y por último, comando3.

Ejemplo:

```
$ date;cal;echo "hola"
```

$ (comando1;comando2;comando3; ...)

Tiene el mismo efecto que el caso anterior pero el resultado de la ejecución de los comandos puede considerarse como un conjunto.

Ejemplo:

$ date;cal > arch1 primero se ejecuta el comando **date**, mostrándose la fecha/hora por pantalla y a continuación se ejecuta el comando cal, guardándose en el archivo arch1 el calendario del año actual

$ (date;cal) > arch1 tanto el resultado del comando **date** como el resultado del comando **cal**, se guardan ambos en el archivo **arch1**

$ (comando1;comando2;comando3; ...) &

Se trata de una ejecución en segundo plano o background. El carácter latino & se llama en inglés **ampersand**. El efecto que tiene es que los comandos se ejecutan y el shell toma inmediatamente el control del sistema, permitiéndonos trabajar sin tener que esperar a que terminen los comandos.

Ejemplo:

$ (rm –r /home;find / -name p* -print)& → mientras se borran recursivamente los archivos y directorios del directorio /home y mientras se buscan archivos que empiecen por p, el usuario puede ejecutar nuevos comandos.

$ rm –r /home;find / -name p* -print & → primero se ejecuta el comando rm –r, y cuando termine empezará la ejecución en segundo plano del comando find; es en ese momento cuando ya se pueden ejecutar nuevos comandos.

$ comando1 && comando2

Comando2 se ejecutará **SI Y SÓLO SI** la ejecución de comando1 ha tenido éxito.

Ejemplo:

$ cat archivo1&&rm archivo5 → si archivo1 existe, y se muestra su contenido por pantalla, entonces se borrará archivo5; si archivo1 no existe o si hubiéramos tecleado mal el comando cat, aparecería un mensaje de error (No such file or directory) y entonces el comando **rm** no se ejecutaría

$ comando1 || comando2

Comando2 se ejecutará **SI Y SÓLO SI** la ejecución de comando1 **NO** ha tenido éxito.

Ejemplo:

$ cat archivo1||echo "NO EXISTE ESE ARCHIVO" → si no existe archivo1, entonces se mostrará el mensaje

$ comando1 | comando2 | comando3 |...

Se trata de una ejecución de comandos de filtro. El primer comando puede ser cualquiera. En cambio, los restantes deben ser comando de filtro, es decir, capaces de aceptar la salida del anterior y enviar dicha salida como entrada del siguiente.

Ejemplo:

$ ls –l |more |sort → muestra en pantalla el listado de archivos/directorios haciendo pausa cuando se llene una pantalla y clasificados alfabéticamente